

JAVA PROGRAMMING LABORATORY

INDUSTRY PROBLEM-SOLVING TASK

Java AWT, Applets, Event Handling and Layout Managers

Name of the Student	E.lakshmi Narayanan	Register No.	192524249
Programme / Section	CO4-AT1-Industry problem solving	Date	23/08/2026
Course Code	CSA0925	Course	Programming in Java
Faculty	DR.Sivasankar	Duration	Lab Session

INDUSTRY TASK 1: Employee Performance & Payroll Dashboard

Problem Statement: A software company wants to develop a simple Employee Performance and Payroll Management System using Java AWT. The application should allow the HR team to enter employee details, process salary information, and generate a structured payroll report.

Functional Requirements

- Develop an AWT-based graphical user interface.
- Accept Employee ID, Employee Name, Department, Designation, Basic Salary, Working Days, and Performance Rating.
- Use suitable AWT components such as Label, TextField, Choice, Checkbox/CheckboxGroup, Button, TextArea, and List.
- Calculate Gross Salary, Performance Bonus, and Net Salary using suitable business rules defined by the student.
- Display the complete employee payroll report in a List or TextArea when Generate Report is clicked.
- Use GridBagLayout or BorderLayout for arranging the interface.
- Implement button events using ActionListener.
- Provide Submit/Generate Report and Reset functionality.
- Validate important inputs such as empty fields and numeric salary values.

Industry Context: The application simulates an HR payroll module used to capture employee information, perform basic payroll processing, and present a report for HR verification.

Expected Deliverables

- Java source code.
- Working GUI application with event handling.
- Sample input and output.
- Screenshot of the final application.
- Brief explanation of components, layout manager, and event handling used.

JAVA PROGRAMMING LABORATORY

Program code:

```
import java.awt.*;
import java.awt.event.*;

public class EmployeePayroll extends Frame implements ActionListener {

    Label idLabel, nameLabel, deptLabel, desigLabel;
    Label salaryLabel, daysLabel, ratingLabel;

    TextField idField, nameField, salaryField, daysField;

    Choice deptChoice, desigChoice;

    CheckboxGroup ratingGroup;
    Checkbox r1, r2, r3, r4, r5;

    Button submitButton, reportButton, resetButton;

    TextArea outputArea;
    List employeeList;

    public EmployeePayroll() {

        setTitle("Employee Performance & Payroll Dashboard");
        setSize(900, 600);
        setLayout(new BorderLayout(10, 10));

        Label heading = new Label(
            "EMPLOYEE PERFORMANCE & PAYROLL DASHBOARD",
            Label.CENTER
        );

        heading.setFont(new Font("Arial", Font.BOLD, 20));

        add(heading, BorderLayout.NORTH);

        Panel formPanel = new Panel();
        formPanel.setLayout(new GridBagLayout());

        GridBagConstraints gbc = new GridBagConstraints();

        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.fill = GridBagConstraints.HORIZONTAL;

        idLabel = new Label("Employee ID:");
```

JAVA PROGRAMMING LABORATORY

```
nameLabel = new Label("Employee Name:");  
deptLabel = new Label("Department:");  
desigLabel = new Label("Designation:");  
salaryLabel = new Label("Basic Salary:");  
daysLabel = new Label("Working Days:");  
ratingLabel = new Label("Performance Rating:");
```

```
idField = new TextField(15);  
nameField = new TextField(15);  
salaryField = new TextField(15);  
daysField = new TextField(15);
```

```
deptChoice = new Choice();
```

```
deptChoice.add("IT");  
deptChoice.add("HR");  
deptChoice.add("Finance");  
deptChoice.add("Marketing");  
deptChoice.add("Sales");
```

```
desigChoice = new Choice();
```

```
desigChoice.add("Software Engineer");  
desigChoice.add("Team Leader");  
desigChoice.add("Manager");  
desigChoice.add("HR Executive");  
desigChoice.add("Accountant");
```

```
ratingGroup = new CheckboxGroup();
```

```
r1 = new Checkbox("1", ratingGroup, false);  
r2 = new Checkbox("2", ratingGroup, false);  
r3 = new Checkbox("3", ratingGroup, true);  
r4 = new Checkbox("4", ratingGroup, false);  
r5 = new Checkbox("5", ratingGroup, false);
```

```
Panel ratingPanel = new Panel();
```

```
ratingPanel.add(r1);  
ratingPanel.add(r2);  
ratingPanel.add(r3);  
ratingPanel.add(r4);  
ratingPanel.add(r5);
```

```
gbc.gridx = 0;  
gbc.gridy = 0;  
formPanel.add(idLabel, gbc);
```

JAVA PROGRAMMING LABORATORY

```
gbc.gridx = 1;  
formPanel.add(idField, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 1;  
formPanel.add(nameLabel, gbc);
```

```
gbc.gridx = 1;  
formPanel.add(nameField, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 2;  
formPanel.add(deptLabel, gbc);
```

```
gbc.gridx = 1;  
formPanel.add(deptChoice, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 3;  
formPanel.add(desigLabel, gbc);
```

```
gbc.gridx = 1;  
formPanel.add(desigChoice, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 4;  
formPanel.add(salaryLabel, gbc);
```

```
gbc.gridx = 1;  
formPanel.add(salaryField, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 5;  
formPanel.add(daysLabel, gbc);
```

```
gbc.gridx = 1;  
formPanel.add(daysField, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 6;  
formPanel.add(ratingLabel, gbc);
```

```
gbc.gridx = 1;  
formPanel.add(ratingPanel, gbc);
```

```
submitButton = new Button("Submit");
```

JAVA PROGRAMMING LABORATORY

```
reportButton = new Button("Generate Report");
resetButton = new Button("Reset");

Panel buttonPanel = new Panel();

buttonPanel.add(submitButton);
buttonPanel.add(reportButton);
buttonPanel.add(resetButton);

gbc.gridx = 0;
gbc.gridy = 7;
gbc.gridwidth = 2;

formPanel.add(buttonPanel, gbc);

add(formPanel, BorderLayout.WEST);

outputArea = new TextArea("", 20, 40);

outputArea.setEditable(false);

Panel outputPanel = new Panel(new BorderLayout());

outputPanel.add(
    new Label("Payroll Report", Label.CENTER),
    BorderLayout.NORTH
);

outputPanel.add(outputArea, BorderLayout.CENTER);

add(outputPanel, BorderLayout.CENTER);

employeeList = new List(10);

Panel listPanel = new Panel(new BorderLayout());

listPanel.add(
    new Label("Employee List", Label.CENTER),
    BorderLayout.NORTH
);

listPanel.add(employeeList, BorderLayout.CENTER);

add(listPanel, BorderLayout.EAST);

submitButton.addActionListener(this);
reportButton.addActionListener(this);
```

JAVA PROGRAMMING LABORATORY

```
resetButton.addActionListener(this);

addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {
        dispose();
    }

});

setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == submitButton) {
        generatePayroll();
    }

    else if (e.getSource() == reportButton) {
        generatePayroll();
    }

    else if (e.getSource() == resetButton) {
        resetFields();
    }
}

public void generatePayroll() {

    String id = idField.getText().trim();
    String name = nameField.getText().trim();
    String salaryText = salaryField.getText().trim();
    String daysText = daysField.getText().trim();

    if (id.isEmpty() || name.isEmpty() ||
        salaryText.isEmpty() || daysText.isEmpty()) {

        outputArea.setText(
            "Please fill all required fields."
        );

        return;
    }

    double basicSalary;
    int workingDays;
```

JAVA PROGRAMMING LABORATORY

```
try {

    basicSalary = Double.parseDouble(salaryText);
    workingDays = Integer.parseInt(daysText);

    if (basicSalary <= 0) {

        outputArea.setText(
            "Basic salary must be greater than 0."
        );

        return;
    }

    if (workingDays <= 0 || workingDays > 26) {

        outputArea.setText(
            "Working days must be between 1 and 26."
        );

        return;
    }

}

catch (NumberFormatException ex) {

    outputArea.setText(
        "Basic Salary and Working Days must be numeric."
    );

    return;
}

String department = deptChoice.getSelectedItemAt();
String designation = desigChoice.getSelectedItemAt();

Checkbox selectedRating =
    ratingGroup.getSelectedCheckbox();

int rating =
    Integer.parseInt(selectedRating.getLabel());

double adjustedSalary =
    (basicSalary / 26) * workingDays;
```

JAVA PROGRAMMING LABORATORY

```
double hra =
    adjustedSalary * 0.20;

double grossSalary =
    adjustedSalary + hra;

double bonusPercentage;

if (rating == 5) {
    bonusPercentage = 0.20;
}

else if (rating == 4) {
    bonusPercentage = 0.15;
}

else if (rating == 3) {
    bonusPercentage = 0.10;
}

else if (rating == 2) {
    bonusPercentage = 0.05;
}

else {
    bonusPercentage = 0.00;
}

double performanceBonus =
    basicSalary * bonusPercentage;

double netSalary =
    grossSalary + performanceBonus;

outputArea.setText("");

outputArea.append(
    "=====\n"
);

outputArea.append(
    "    EMPLOYEE PAYROLL REPORT\n"
);

outputArea.append(
    "=====\n"
);
```

JAVA PROGRAMMING LABORATORY

```
outputArea.append(
    "Employee ID    : " + id + "\n"
);

outputArea.append(
    "Employee Name  : " + name + "\n"
);

outputArea.append(
    "Department    : " + department + "\n"
);

outputArea.append(
    "Designation    : " + designation + "\n"
);

outputArea.append(
    "Basic Salary   : " +
    String.format("%.2f", basicSalary) + "\n"
);

outputArea.append(
    "Working Days   : " + workingDays + "\n"
);

outputArea.append(
    "Performance Rating: " + rating + "\n"
);

outputArea.append(
    "-----\n"
);

outputArea.append(
    "Adjusted Salary : " +
    String.format("%.2f", adjustedSalary) + "\n"
);

outputArea.append(
    "HRA (20%)      : " +
    String.format("%.2f", hra) + "\n"
);

outputArea.append(
    "Gross Salary   : " +
    String.format("%.2f", grossSalary) + "\n"
```

JAVA PROGRAMMING LABORATORY

```
);

outputArea.append(
    "Performance Bonus : " +
    String.format("%.2f", performanceBonus) + "\n"
);

outputArea.append(
    "Net Salary      : " +
    String.format("%.2f", netSalary) + "\n"
);

outputArea.append(
    "=====\n"
);

employeeList.add(
    id + " - " + name +
    " - " + department +
    " - Rs. " +
    String.format("%.2f", netSalary)
);
}

public void resetFields() {

    idField.setText("");
    nameField.setText("");
    salaryField.setText("");
    daysField.setText("");

    deptChoice.select(0);
    desigChoice.select(0);

    ratingGroup.setSelectedCheckbox(r3);

    outputArea.setText("");
}

public static void main(String[] args) {

    new EmployeePayroll();
}
}
```

JAVA PROGRAMMING LABORATORY

The screenshot shows a Java Swing window titled "Employee Performance & Payroll Dashboard". The window is divided into three main sections:

- Left Section (Form):** Contains input fields for Employee ID (1), Employee Name (mani), Department (IT), Designation (Software Engineer), Basic Salary (50000), and Working Days (5). Below these is a Performance Rating section with radio buttons for 1, 2, 3 (selected), 4, and 5. At the bottom are three buttons: "Submit", "Generate Report", and "Reset".
- Center Section (Payroll Report):** Titled "Payroll Report", it displays an "EMPLOYEE PAYROLL REPORT" for Employee ID 1, Name mani, Department IT, Designation Software Engineer, Basic Salary 50000.00, Working Days 5, and Performance Rating 3. The report shows calculated values: Adjusted Salary (9615.38), HRA (20%) (1923.08), Gross Salary (11538.46), Performance Bonus (5000.00), and Net Salary (16538.46).
- Right Section (Employee List):** Titled "Employee List", it shows a list of employees: "1 - narayanan - IT - Rs. 16538.46", "1 - narayanan - IT - Rs. 16538.46", and "1 - mani - IT - Rs. 16538.46".

INDUSTRY TASK 2: Online Food Ordering and Billing System

Problem Statement: A restaurant wants to develop a simple Food Ordering and Billing Application for its counter staff. The application should allow customers to select food items, specify quantities, and generate a final bill.

Functional Requirements

- Develop the application using Java AWT.
- Create fields for Customer Name and Table Number.
- Provide Food Category, Food Item, and Quantity selection using appropriate AWT components.
- Provide Add Item, Generate Bill, Clear, Next, and Previous buttons.
- Display selected food items in a List or TextArea.
- Calculate item-wise cost, Subtotal, GST, and Final Bill Amount.
- Implement button click events using ActionListener.
- Use CardLayout to create separate screens for Customer/Order Details, Food Selection, and Bill/Order Summary.
- Validate quantity and ensure that an item is selected before adding it to the order.

Industry Context: The application represents a simplified restaurant POS (Point of Sale) system used for taking customer orders, processing prices, and generating a customer bill.

Expected Deliverables

JAVA PROGRAMMING LABORATORY

- Java source code.
- Working multi-card AWT interface.
- Sample order and generated bill.
- Screenshot of each card/screen.
- Brief explanation of CardLayout, AWT components, and event handling.

Program code

```
import java.awt.*;
import java.awt.event.*;

public class FoodOrderingSystem extends Frame implements ActionListener {

    CardLayout cardLayout;
    Panel cardPanel;

    TextField customerNameField;
    TextField tableNumberField;

    Choice categoryChoice;
    Choice foodChoice;
    TextField quantityField;

    List orderList;

    Button addItemButton;
    Button generateBillButton;
    Button clearButton;
    Button nextButton;
    Button previousButton;

    Button billPreviousButton;
    Button billClearButton;

    TextArea billArea;

    double subtotal = 0.0;

    final double GST_RATE = 0.05;

    String[] foodNames = new String[50];
    int[] foodQuantities = new int[50];
    double[] foodPrices = new double[50];

    int itemCount = 0;

    public FoodOrderingSystem() {
```

JAVA PROGRAMMING LABORATORY

```
setTitle("Online Food Ordering and Billing System");  
setSize(900, 600);  
setLayout(new BorderLayout());
```

```
cardLayout = new CardLayout();  
cardPanel = new Panel(cardLayout);
```

```
Panel customerPanel = new Panel(new GridBagLayout());
```

```
GridBagConstraints gbc = new GridBagConstraints();
```

```
gbc.insets = new Insets(10, 10, 10, 10);  
gbc.fill = GridBagConstraints.HORIZONTAL;
```

```
Label title1 = new Label(  
    "CUSTOMER / ORDER DETAILS",  
    Label.CENTER  
);
```

```
title1.setFont(  
    new Font("Arial", Font.BOLD, 22)  
);
```

```
Label customerLabel =  
    new Label("Customer Name:");
```

```
Label tableLabel =  
    new Label("Table Number:");
```

```
customerNameField =  
    new TextField(20);
```

```
tableNumberField =  
    new TextField(20);
```

```
nextButton =  
    new Button("Next");
```

```
gbc.gridx = 0;  
gbc.gridy = 0;  
gbc.gridwidth = 2;
```

```
customerPanel.add(title1, gbc);
```

```
gbc.gridwidth = 1;
```

JAVA PROGRAMMING LABORATORY

```
gbc.gridx = 0;  
gbc.gridy = 1;
```

```
customerPanel.add(  
    customerLabel,  
    gbc  
);
```

```
gbc.gridx = 1;
```

```
customerPanel.add(  
    customerNameField,  
    gbc  
);
```

```
gbc.gridx = 0;  
gbc.gridy = 2;
```

```
customerPanel.add(  
    tableLabel,  
    gbc  
);
```

```
gbc.gridx = 1;
```

```
customerPanel.add(  
    tableNumberField,  
    gbc  
);
```

```
gbc.gridx = 0;  
gbc.gridy = 3;  
gbc.gridwidth = 2;
```

```
customerPanel.add(  
    nextButton,  
    gbc  
);
```

```
Panel foodPanel =  
    new Panel(new BorderLayout(10, 10));
```

```
Panel foodForm =  
    new Panel(new GridBagLayout());
```

JAVA PROGRAMMING LABORATORY

```
Label title2 = new Label(  
    "FOOD SELECTION",  
    Label.CENTER  
);  
  
title2.setFont(  
    new Font("Arial", Font.BOLD, 22)  
);  
  
Label categoryLabel =  
    new Label("Food Category:");  
  
Label foodLabel =  
    new Label("Food Item:");  
  
Label quantityLabel =  
    new Label("Quantity:");  
  
categoryChoice =  
    new Choice();  
  
categoryChoice.add("South Indian");  
categoryChoice.add("North Indian");  
categoryChoice.add("Chinese");  
categoryChoice.add("Fast Food");  
categoryChoice.add("Drinks");  
  
foodChoice =  
    new Choice();  
  
quantityField =  
    new TextField(10);  
  
addItemButton =  
    new Button("Add Item");  
  
generateBillButton =  
    new Button("Generate Bill");  
  
previousButton =  
    new Button("Previous");  
  
clearButton =  
    new Button("Clear");  
  
GridBagConstraints fgbc =
```

JAVA PROGRAMMING LABORATORY

```
new GridBagConstraints();

fgbc.insets =
    new Insets(8, 8, 8, 8);

fgbc.fill =
    GridBagConstraints.HORIZONTAL;

fgbc.gridx = 0;
fgbc.gridy = 0;
fgbc.gridwidth = 2;

foodForm.add(
    title2,
    fgbc
);

fgbc.gridwidth = 1;

fgbc.gridx = 0;
fgbc.gridy = 1;

foodForm.add(
    categoryLabel,
    fgbc
);

fgbc.gridx = 1;

foodForm.add(
    categoryChoice,
    fgbc
);

fgbc.gridx = 0;
fgbc.gridy = 2;

foodForm.add(
    foodLabel,
    fgbc
);

fgbc.gridx = 1;

foodForm.add(
    foodChoice,
    fgbc
```

JAVA PROGRAMMING LABORATORY

```
);

fgbc.gridx = 0;
fgbc.gridy = 3;

foodForm.add(
    quantityLabel,
    fgbc
);

fgbc.gridx = 1;

foodForm.add(
    quantityField,
    fgbc
);

Panel buttonPanel =
    new Panel(new FlowLayout());

buttonPanel.add(
    addItemButton
);

buttonPanel.add(
    generateBillButton
);

buttonPanel.add(
    previousButton
);

buttonPanel.add(
    clearButton
);

fgbc.gridx = 0;
fgbc.gridy = 4;
fgbc.gridwidth = 2;

foodForm.add(
    buttonPanel,
    fgbc
);

foodPanel.add(
    foodForm,
```

JAVA PROGRAMMING LABORATORY

```
        BorderLayout.NORTH
    );

    orderList =
        new List(15);

    Panel listPanel =
        new Panel(new BorderLayout());

    listPanel.add(
        new Label(
            "SELECTED FOOD ITEMS",
            Label.CENTER
        ),
        BorderLayout.NORTH
    );

    listPanel.add(
        orderList,
        BorderLayout.CENTER
    );

    foodPanel.add(
        listPanel,
        BorderLayout.CENTER
    );

    Panel billPanel =
        new Panel(new BorderLayout(10, 10));

    Label title3 = new Label(
        "BILL / ORDER SUMMARY",
        Label.CENTER
    );

    title3.setFont(
        new Font("Arial", Font.BOLD, 22)
    );

    billArea =
        new TextArea();

    billArea.setEditable(false);

    billPreviousButton =
        new Button("Previous");
```

JAVA PROGRAMMING LABORATORY

```
billClearButton =  
    new Button("Clear");
```

```
Panel billButtonPanel =  
    new Panel(new FlowLayout());
```

```
billButtonPanel.add(  
    billPreviousButton  
);
```

```
billButtonPanel.add(  
    billClearButton  
);
```

```
billPanel.add(  
    title3,  
    BorderLayout.NORTH  
);
```

```
billPanel.add(  
    billArea,  
    BorderLayout.CENTER  
);
```

```
billPanel.add(  
    billButtonPanel,  
    BorderLayout.SOUTH  
);
```

```
cardPanel.add(  
    customerPanel,  
    "Customer"  
);
```

```
cardPanel.add(  
    foodPanel,  
    "Food"  
);
```

```
cardPanel.add(  
    billPanel,  
    "Bill"  
);
```

```
add(  
    cardPanel,
```

JAVA PROGRAMMING LABORATORY

```
        BorderLayout.CENTER
    );

    nextButton.addActionListener(this);

    addItemButton.addActionListener(this);

    generateBillButton.addActionListener(this);

    previousButton.addActionListener(this);

    clearButton.addActionListener(this);

    billPreviousButton.addActionListener(this);

    billClearButton.addActionListener(this);

    // Category selection event

    categoryChoice.addItemListener(
        new ItemListener() {

            public void itemStateChanged(
                ItemEvent e
            ) {

                updateFoodItems();
            }
        }
    );

    updateFoodItems();

    // Window closing event

    addWindowListener(
        new WindowAdapter() {

            public void windowClosing(
                WindowEvent e
            ) {

                dispose();
            }
        }
    );
```

JAVA PROGRAMMING LABORATORY

```
setVisible(true);
}

public void updateFoodItems() {

    foodChoice.removeAll();

    String category =
        categoryChoice.getSelectedItem();

    if (category.equals("South Indian")) {

        foodChoice.add("Idli - 50");
        foodChoice.add("Dosa - 80");
        foodChoice.add("Pongal - 70");
        foodChoice.add("Poori - 60");
    }

    else if (category.equals("North Indian")) {

        foodChoice.add("Chapati - 60");
        foodChoice.add("Paneer Butter Masala - 150");
        foodChoice.add("Veg Thali - 180");
        foodChoice.add("Dal Tadka - 120");
    }

    else if (category.equals("Chinese")) {

        foodChoice.add("Veg Noodles - 120");
        foodChoice.add("Fried Rice - 130");
        foodChoice.add("Manchurian - 140");
        foodChoice.add("Spring Roll - 100");
    }

    else if (category.equals("Fast Food")) {

        foodChoice.add("Burger - 120");
        foodChoice.add("Pizza - 250");
        foodChoice.add("French Fries - 90");
        foodChoice.add("Sandwich - 100");
    }

    else if (category.equals("Drinks")) {

        foodChoice.add("Tea - 30");
```

JAVA PROGRAMMING LABORATORY

```
        foodChoice.add("Coffee - 40");
        foodChoice.add("Fresh Juice - 80");
        foodChoice.add("Milkshake - 100");
    }
}

// =====
// GET PRICE
// =====

public double getPrice(String item) {

    int dashPosition =
        item.lastIndexOf("-");

    String price =
        item.substring(
            dashPosition + 1
        ).trim();

    return Double.parseDouble(price);
}

// =====
// GET FOOD NAME
// =====

public String getFoodName(String item) {

    int dashPosition =
        item.lastIndexOf("-");

    return item.substring(
        0,
        dashPosition
    ).trim();
}

// =====
// ACTION LISTENER
// =====

public void actionPerformed(
    ActionEvent e
) {

    String command =
```

JAVA PROGRAMMING LABORATORY

```
e.getActionCommand();

// Next button

if (command.equals("Next")) {

    String customer =
        customerNameField
            .getText()
            .trim();

    String table =
        tableNumberField
            .getText()
            .trim();

    if (customer.isEmpty() ||
        table.isEmpty()) {

        showMessage(
            "Please enter Customer Name and Table Number."
        );

        return;
    }

    try {

        int tableNumber =
            Integer.parseInt(table);

        if (tableNumber <= 0) {

            showMessage(
                "Enter a valid Table Number."
            );

            return;
        }
    }

    catch (NumberFormatException ex) {

        showMessage(
            "Table Number must be numeric."
        );
    }
}
```

JAVA PROGRAMMING LABORATORY

```
        return;
    }

    cardLayout.show(
        cardPanel,
        "Food"
    );
}

// Add Item button

else if (command.equals("Add Item")) {

    addFoodItem();
}

// Generate Bill button

else if (command.equals("Generate Bill")) {

    if (itemCount == 0) {

        showMessage(
            "Please add at least one food item."
        );

        return;
    }

    generateBill();

    cardLayout.show(
        cardPanel,
        "Bill"
    );
}

// Previous button

else if (command.equals("Previous")) {

    cardLayout.show(
        cardPanel,
        "Customer"
    );
}
```

JAVA PROGRAMMING LABORATORY

```
// Clear button

else if (command.equals("Clear")) {

    clearOrder();
}

// Bill Previous button

else if (command.equals("BillPrevious")) {

    cardLayout.show(
        cardPanel,
        "Food"
    );
}

// Bill Clear button

else if (command.equals("BillClear")) {

    clearOrder();

    cardLayout.show(
        cardPanel,
        "Customer"
    );
}
}

public void addFoodItem() {

    if (foodChoice.getItemCount() == 0) {

        showMessage(
            "Please select a food item."
        );

        return;
    }

    String quantityText =
        quantityField
            .getText()
            .trim();

    if (quantityText.isEmpty()) {
```

JAVA PROGRAMMING LABORATORY

```
        showMessage(  
            "Please enter quantity."  
        );  
  
        return;  
    }  
  
    int quantity;  
  
    try {  
  
        quantity =  
            Integer.parseInt(  
                quantityText  
            );  
  
        if (quantity <= 0) {  
  
            showMessage(  
                "Quantity must be greater than 0."  
            );  
  
            return;  
        }  
    }  
  
    catch (NumberFormatException ex) {  
  
        showMessage(  
            "Quantity must be a number."  
        );  
  
        return;  
    }  
  
    String selectedItem =  
        foodChoice.getSelectedItemAt();  
  
    double price =  
        getPrice(selectedItem);  
  
    String foodName =  
        getFoodName(selectedItem);  
  
    double itemCost =  
        price * quantity;
```

JAVA PROGRAMMING LABORATORY

```
foodNames[itemCount] =  
    foodName;  
  
foodQuantities[itemCount] =  
    quantity;  
  
foodPrices[itemCount] =  
    price;  
  
itemCount++;  
  
subtotal += itemCost;  
  
orderList.add(  
    foodName +  
    " x " +  
    quantity +  
    " = Rs." +  
    String.format(  
        "%.2f",  
        itemCost  
    )  
);  
  
quantityField.setText("");  
  
showMessage(  
    "Item added successfully."  
);  
}  
  
public void generateBill() {  
  
    double gst =  
        subtotal * GST_RATE;  
  
    double finalAmount =  
        subtotal + gst;  
  
    String customer =  
        customerNameField.getText();  
  
    String table =  
        tableNumberField.getText();
```

JAVA PROGRAMMING LABORATORY

```
billArea.setText("");

billArea.append(
    "=====\n"
);

billArea.append(
    "      RESTAURANT BILL\n"
);

billArea.append(
    "=====\n"
);

billArea.append(
    "Customer Name : " +
    customer +
    "\n"
);

billArea.append(
    "Table Number : " +
    table +
    "\n"
);

billArea.append(
    "-----\n"
);

billArea.append(
    "Food Item      Qty      Amount\n"
);

billArea.append(
    "-----\n"
);

for (int i = 0; i < itemCount; i++) {

    double amount =
        foodQuantities[i] *
        foodPrices[i];

    billArea.append(
        foodNames[i] +
        "      " +
```

JAVA PROGRAMMING LABORATORY

```
        foodQuantities[i] +  
        "    Rs." +  
        String.format(  
            "%.2f",  
            amount  
        ) +  
        "\n"  
    );  
}  
  
billArea.append(  
    "-----\n"  
);  
  
billArea.append(  
    "Subtotal   : Rs." +  
    String.format(  
        "%.2f",  
        subtotal  
    ) +  
    "\n"  
);  
  
billArea.append(  
    "GST (5%)   : Rs." +  
    String.format(  
        "%.2f",  
        gst  
    ) +  
    "\n"  
);  
  
billArea.append(  
    "Final Amount : Rs." +  
    String.format(  
        "%.2f",  
        finalAmount  
    ) +  
    "\n"  
);  
  
billArea.append(  
    "=====\n"  
);  
  
billArea.append(  
    "    THANK YOU! VISIT AGAIN\n"
```

JAVA PROGRAMMING LABORATORY

```
);  
  
billArea.append(  
    "=====\n"  
);  
}
```

```
public void clearOrder() {  
  
    customerNameField.setText("");  
  
    tableNumberField.setText("");  
  
    quantityField.setText("");  
  
    orderList.removeAll();  
  
    billArea.setText("");  
  
    subtotal = 0.0;  
  
    itemCount = 0;  
  
    cardLayout.show(  
        cardPanel,  
        "Customer"  
    );  
}
```

```
public void showMessage(  
    String message  
) {
```

```
    Dialog dialog =  
        new Dialog(  
            this,  
            "Message",  
            true  
        );
```

```
    dialog.setLayout(  
        new BorderLayout()  
    );
```

```
    Label messageLabel =  
        new Label(  

```

JAVA PROGRAMMING LABORATORY

```
        message,
        Label.CENTER
    );

    Button okButton =
        new Button("OK");

    okButton.addActionListener(
        new ActionListener() {

            public void actionPerformed(
                ActionEvent e
            ) {

                dialog.dispose();
            }
        }
    );

    dialog.add(
        messageLabel,
        BorderLayout.CENTER
    );

    dialog.add(
        okButton,
        BorderLayout.SOUTH
    );

    dialog.setSize(
        400,
        150
    );

    dialog.setLocationRelativeTo(
        this
    );

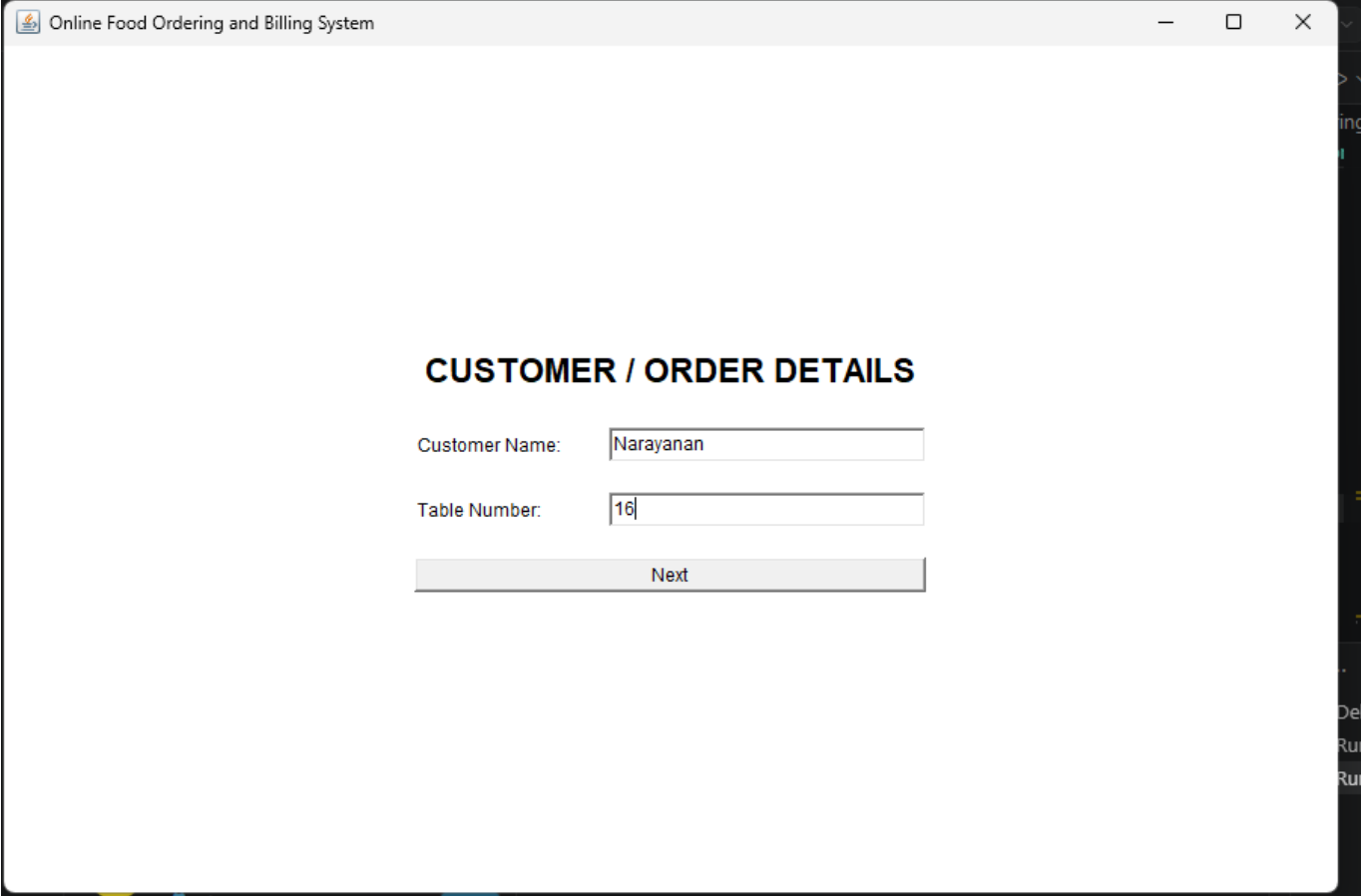
    dialog.setVisible(true);
}

public static void main(
    String[] args
) {

    new FoodOrderingSystem();
}
```

JAVA PROGRAMMING LABORATORY

```
}  
}
```



The screenshot shows a Java AWT window titled "Online Food Ordering and Billing System". The window contains a form titled "CUSTOMER / ORDER DETAILS". The form has two input fields: "Customer Name" with the value "Narayanan" and "Table Number" with the value "16". Below the input fields is a "Next" button.

Online Food Ordering and Billing System

CUSTOMER / ORDER DETAILS

Customer Name:

Table Number:

JAVA PROGRAMMING LABORATORY

Online Food Ordering and Billing System

FOOD SELECTION

Food Category:

Food Item:

Quantity:

Dosa x 2 = Rs.160.00
Dosa x 3 = Rs.240.00
Dosa x 4 = Rs.320.00

Message

Item added successfully.

OK

JAVA PROGRAMMING LABORATORY

Online Food Ordering and Billing System

BILL / ORDER SUMMARY

=====

RESTAURANT BILL

=====

Customer Name : Narayanan
Table Number : 16

=====

Food Item	Qty	Amount
Dosa	2	Rs.160.00
Dosa	3	Rs.240.00
Dosa	4	Rs.320.00

=====

Subtotal : Rs.720.00
GST (5%) : Rs.36.00
Final Amount : Rs.756.00

=====

THANK YOU! VISIT AGAIN

=====

Previous Clear

Evaluation Rubric

Criterion	Weightage	Marks / Remarks
Problem Understanding	15%	
GUI Components and Layout	20%	
Event Handling	20%	
Functional Correctness	25%	
Input Validation and User Experience	10%	
Program Explanation / Viva	10%	

Student Instructions

1. The solution must be implemented in Java using the concepts covered in the laboratory syllabus.
2. Students should demonstrate compilation, execution, and output during the practical session.
3. Use meaningful variable names and appropriate comments.
4. Students must be able to explain the selected AWT components, layout manager, event handling mechanism, and processing logic.